Object Oriented Programming [FECEg-3142]

CHAPTER FOUR:

Inheritance

Outline

- Introduction
- Superclasses and Subclasses
- Protected members
- Constructors in Subclasses

Introduction

- Object-oriented programming allows you to derive new classes from existing classes, this is called **inheritance**.
- **Inheritance** is an important and powerful feature in Java for reusing software.
- Suppose you are to define classes to model circles, rectangles, and triangles.
  - These classes have many common features.
- What is the best way to design these classes so to avoid redundancy and make the system easy to comprehend and easy to maintain?
  - The answer is to use inheritance.

Superclasses and Subclasses

- We can use a class to model objects of the same type.
- Different classes may have some common properties and behaviors, which can be generalized in a class that can be shared by other classes.
- Inheritance enables us to define a general class and later **extend** it to more specialized classes.
- The **specialized classes** inherit the **properties** and **methods** from the **general class**.

Cont'd...

- A **superclass** is also referred to as a **parent class**, or a **base class**, and a **subclass** as a **child class**, an **extended class**, or a **derived class**.
- A **subclass** inherits accessible **data fields** and **methods** from its **superclass** and may also add new **data fields** and **methods**.
- **extends** is the keyword used to inherit the properties of a class.

Cont'd...

- Following is the **syntax** of extends keyword.

```
class Super {
   .....
   .....
}
class Sub extends Super {
   .....
   .....
}
```

Example

```
package inheritanceexample;
public class Person {
     private String name;
    public Person(){
     name = "No name yet";
    }
    public Person(String initialName){
     name = initialName;
    }
    public void setName(String newName){
     name = newName;
    }
    public String getName(){
     return name;
    }
    public void writeOutput(){
     System.out.println("Name: " + name);
    }
    public boolean hasSameName(Person otherPerson){
     return this.name.equalsIgnoreCase(otherPerson.name);
    }
    public boolean equals(Student otherStudent){
     return this.hasSameName(otherStudent) &&
     (this.studentNumber == otherStudent.studentNumber);
    }
}
```

```
package inheritanceexample;
public class Student extends Person{
       private int studentNumber;
    public Student(){
     super();
     studentNumber = 0;//Indicating no number yet
    }
    public Student(String initialName, int initialStudentNumber){
     super(initialName);
     studentNumber = initialNumber;
    }
    public void reset(String newName, int newStudentNumber){
     setName(newName);
     studentNumber = newStudentNumber;
    }
    public int getStudentNumber(){
     return studentNumber;
    }
    public void setStudentNumber(int newStudentNumber){
     studentNumber = newStudentNumber;
    }
    public void writeOutput(){
     System.out.println("Name: " + getName());
     System.out.println("Student Number: " + studentNumber);
    }

}
```

Example

```
package inheritanceexample;

public class InheritanceDemo {
     public static void main(String[] args){
         Student s = new Student();
         s.setName("Warren Peace");
         s.setStudentNumber(1234);
         s.writeOutput();

    }

}
```

Out put run:
Name: Warren Peace
Student Number: 1234

Cont'd...

The following points regarding inheritance are worthwhile to note:

- Contrary to the conventional interpretation, a subclass is not a subset of its superclass.
  - In fact, a subclass usually contains more **information** and **methods** than its superclass.
- **Private data** fields in a superclass are not accessible outside the class.
  - Therefore, they cannot be used directly in a subclass.
  - They can, however, be accessed/mutated through public accessor/mutator if defined in the superclass.

Cont'd...

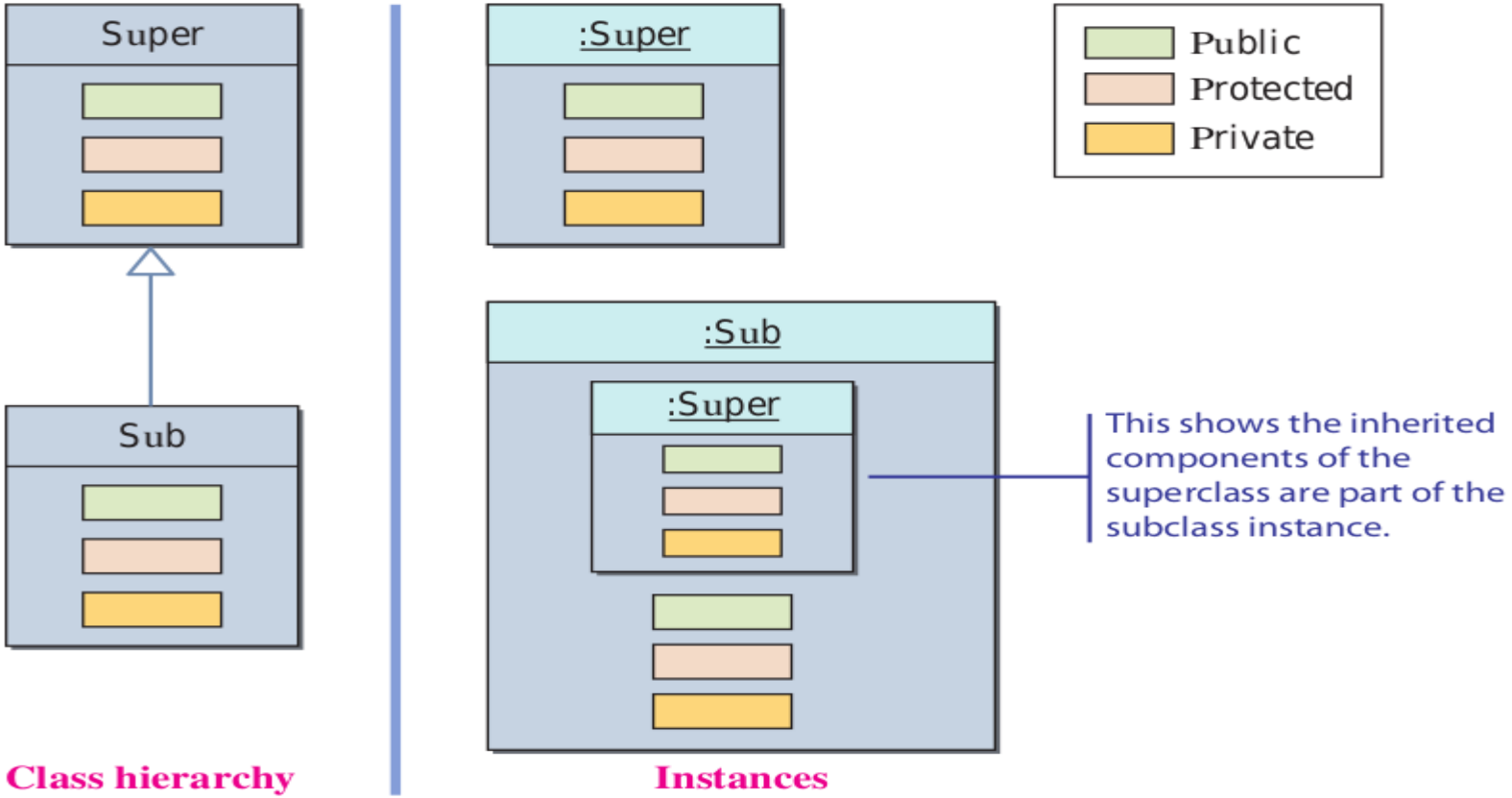
- Not all is-a relationships should be modeled using inheritance.
- Inheritance is used to model the is-a relationship.
- Some programming languages allow you to derive a subclass from several classes.
  - This capability is known as multiple inheritance.
  - Java, however, does not allow multiple inheritance.
  - A Java class may inherit directly from only one superclass.
  - This restriction is known as single inheritance.
  - If you use the extends keyword to define a subclass, it allows only one parent class.
  - Nevertheless, multiple inheritance can be achieved through interfaces.

Inheritance and Member Accessibility

Cont'd...

- Which **members** (**variables** and **methods**) of a superclass are inherited by a subclass and how these members are accessed?
- In addition to declaring **members** **private** and **public** , we can declare them **protected** .
- The **protected modifier** is meaningful when used with inheritance.
- Consider the following declarations:

Cont'd...



A graphical representation of superclasses and subclasses with public, private, and protected members.

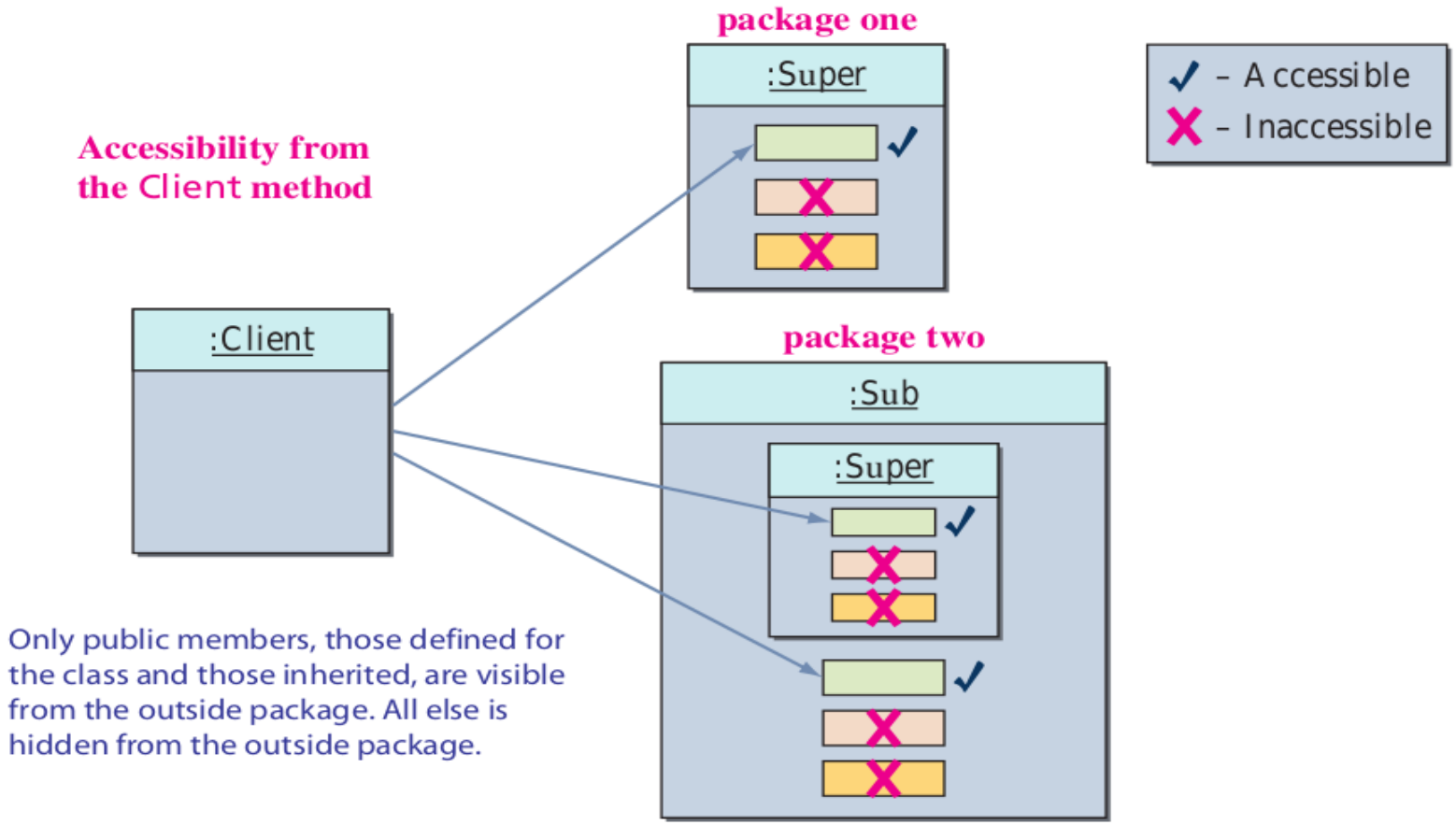
Protected members

- A **public** member is accessible from everywhere, but a private member is accessible only within the same class.
  - Public members of a class, whether they are inherited or not, are accessible from any object or class.
- **Private** members of a class, on the other hand, are never accessible from any outside object or class.
- A **protected** member can be accessed from the classes in the **same package** and the **subclasses** in other packages.
  - It is not accessible from the unrelated classes in other packages.

Cont'd...

- The accessibility of class members from the methods of a Sub object.
  - A method in the **Sub object** can access both the **protected** and **public** members of **Super** , but not the **private** members of **Super** .

Cont'd...



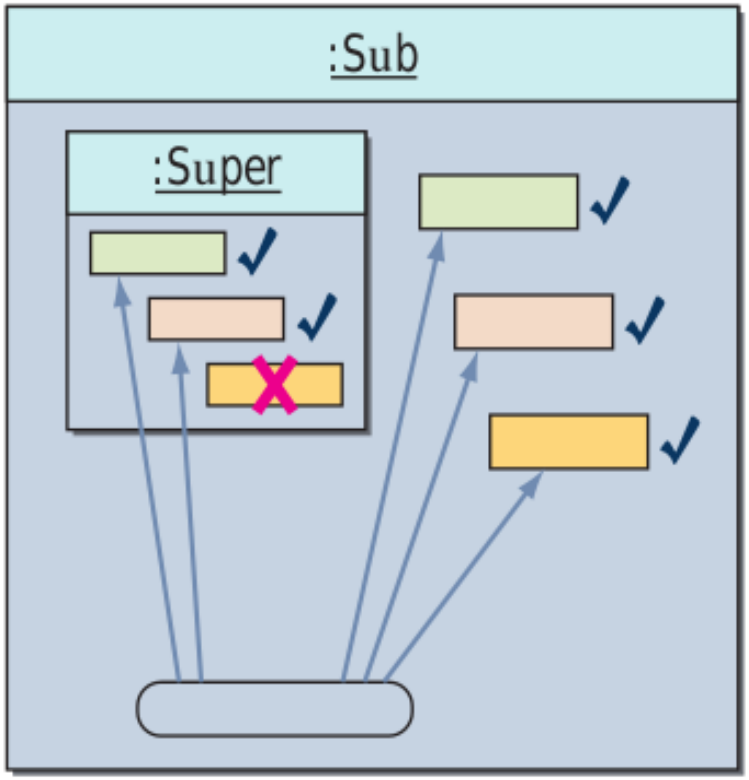
- The difference between **public**, **private**, and **protected** modifiers.
- Only **public** members are visible from outside.

Cont'd...

Accessibility from a method of the Sub class

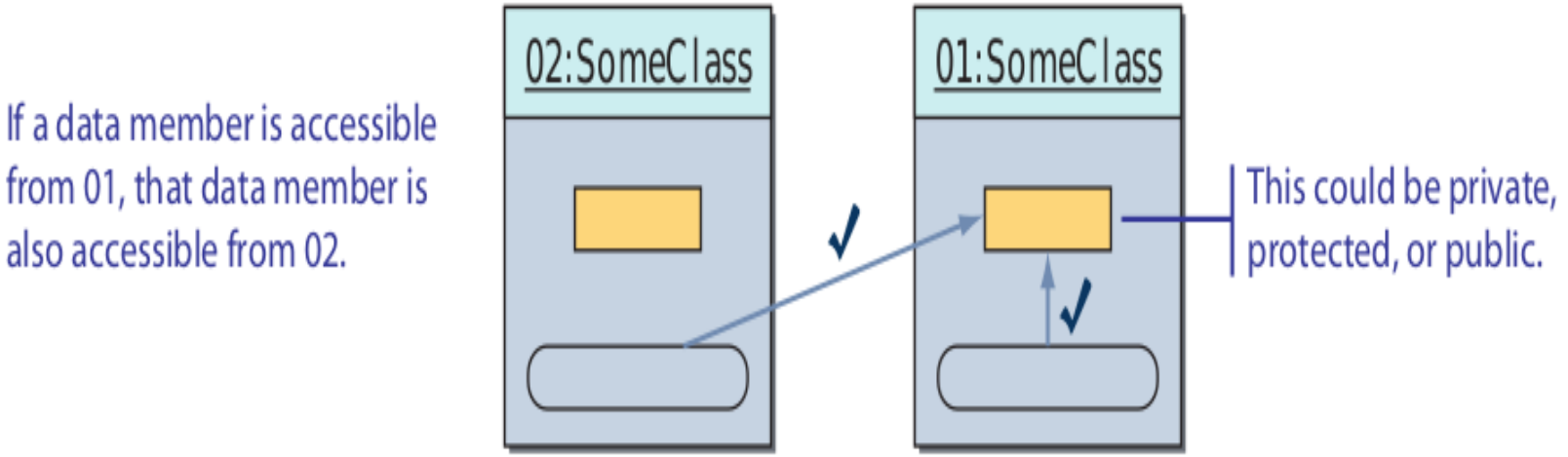
✓ - Accessible
✗ - Inaccessible

From a method of Sub, everything is visible except the private members of its superclass.



- The difference between public, private, and protected modifiers.
- Everything **except the private** members of the **Super class** is **visible** from a **method** of the **Sub class**.

Cont'd...



- Data members accessible from an instance are also accessible from other instances of the same class.

Constructors in Subclasses

- It does not inherit any constructors from the base class.
- In the definition of a constructor for the derived class, the typical first action is to call a constructor of the base class.

Cont'd...

- Consider the following constructor definition in the derived class Student:

```
public Student(String initialName, int initialStudentNumber){
super(initialName);
studentNumber = initialStudentNumber;
}
```

- This constructor uses the reserved word **super** as a method name to call a constructor of the base class.

Cont'd...

- The use of super involves some details: It must always be the first action taken in a constructor definition.
- If you do not include an explicit call to the base-class constructor in any constructor for a derived class, Java will automatically include a call to the base class's default constructor.
- For example, the definition of the default constructor for the class Student

```
public Student(){
   super();
   studentNumber = 0;//Indicating no
   number yet
}
```

```
public Student(){
   studentNumber =
   0;//Indicating no number yet
}
```

- Both are completely equivalent

Cont'd...

- When you omit a call to the base-class constructor in any derived-class constructor, the default constructor of the base class is called as the first action in the new constructor.
- This default constructor—the one without parameters might not be the one that should be called. Thus, including your own call to the base-class constructor is often a good idea.

Cont'd...

The this keyword

- Another common action when defining a constructor is to call another constructor in the same class, using the keyword this .
- We can revise the default constructor in the class Person call another constructor in that class by using this , as follows:

```
public Person(){
   this("No name yet");
}
```

```
public Person(String initialName){
name = initialName;
}
```

In this way, the default constructor calls the constructor thereby setting the instance variable name to the string "No name yet" .

Cont'd...

Overriding Methods

- A subclass inherits methods from a superclass.
  - Sometimes it is necessary for the subclass to modify the implementation of a method defined in the superclass.
  - This is referred to as method overriding.

Cont'd...

Several points are worth noting:

- An instance method can be overridden only if it is accessible.
- Thus a private method cannot be overridden, because it is not accessible outside its own class.
- If a method defined in a subclass is private in its superclass, the two methods are completely unrelated.
- Like an instance method, a static method can be inherited. However, a static method cannot be overridden.
- If a static method defined in the superclass is redefined in a subclass, the method defined in the superclass is hidden.
- The hidden static methods can be invoked using the syntax SuperClassName.staticMethodName .

Cont'd...

Overriding vs. Overloading

- Overloading means to define multiple methods with the same name but different signatures.
- Overriding means to provide a new implementation for a method in the subclass.
- The method is already defined in the superclass.
- To override a method, the method must be defined in the subclass using the same signature and the same return type.

Cont'd...

- Let us use an example to show the differences between overriding and overloading.
- In (a) below, the method p(double i) in class A overrides the same method defined in class B .
- In (b), however, the class B has two overloaded methods p(double i) and p(int i) . The method p(double i) is inherited from B .

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
    a.p(10.0);
  }
}

class B {
  public void p(double i) {
    System.out.println(i * 2);
  }
}

class A extends B {
  // This method overrides the method in B
  public void p(double i) {
    System.out.println(i);
  }
}
```

(a)

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
    a.p(10.0);
  }
}

class B {
  public void p(double i) {
    System.out.println(i * 2);
  }
}

class A extends B {
  // This method overloads the method in B
  public void p(int i) {
    System.out.println(i);
  }
}
```

(b)

Example

```
public class TestMethodOverloading {
    public static void main(String[] args) {
      System.out.println("The maximum between 3 and 4 is "+ max(3, 4) );
      System.out.println("The maximum between 3.0 and 5.4 is " + max(3.0, 5.4) );
      System.out.println("The maximum between 3.0, 5.4, and 10.14 is " + max(3.0, 5.4, 10.14) );
    }
    /** Return the max between two int values */
    public static int max(int num1, int num2) {
      if (num1 > num2)
        return num1;
      else
        return num2;
    }
    /** Find the max between two double values */
    public static double max(double num1, double num2) {
      if (num1 > num2)
        return num1;
      else
        return num2;
    }
    /** Return the max among three double values */
    public static double max(double num1, double num2, double num3) {
      return max(max(num1, num2), num3);
    }
}
```

End